WELT 5 · DOPPELSTUNDEN 34 UND 35 · LEVEL 5.4

W5·2 – Der Wald: Suchbäume

Datei: `AQ_W5_L5_Wald_Geruest.txt` (Klasse Suchbaum, Testklasse L54SuchbaumTest) · Grafik: `AQ_W5_Baumzeichner_Geruest.txt` (Aufgaben 4 bis 8: Suchweg und Item-Datenbank) · Im Lab (Algo Quest Lab): Level 5.4 und 5.5 · Teil 1 in DS 34, Teil 2 in DS 35

Euer Inventar wächst: Mit jedem Raum, den euer Team erobert, kommen neue Items dazu. Eine Reihung durchsucht ihr Element für Element – bei 1000 Items sind das bis zu 1000 Vergleiche, bei jedem Öffnen der Truhe. Die Waldläufer kennen einen Trick: An jeder Weggabelung steht ein Wegweiser „kleinere Nummern links, größere rechts". Wer ihm folgt, muss nur **einen** Weg von der Wurzel nach unten gehen.

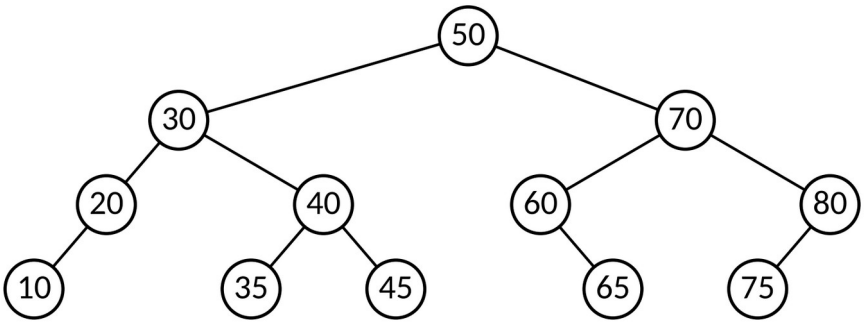
Dieser Baum heißt **binärer Suchbaum**. Er ist das Herz der **Item-Datenbank** eures Spiels – und im Abitur ein Dauerbrenner: Suchbaum zeichnen, Suchweg angeben, Einfügen und Löschen von Hand, Aufwand beurteilen und rekursive Methoden wie suche oder istBST schreiben.

Teil 1 – Suchen, Einfügen, Löschen (DS 34)

Infoteil 1: Der binäre Suchbaum

Ein Binärbaum vom Inhaltstyp Ganzzahl ist ein **binärer Suchbaum**, wenn für **jeden** Knoten gilt: **Alle** Werte in seinem linken Teilbaum sind **kleiner**, **alle** Werte in seinem rechten Teilbaum sind **größer** als sein eigener Wert. Jeder Wert kommt höchstens einmal vor – ein Duplikat wird beim Einfügen ignoriert.

- Folge 1: Die **Inorder** eines Suchbaums ist **aufsteigend sortiert**.
- Folge 2: Das **Minimum** steht ganz links (immer links, bis es nicht weitergeht), das Maximum ganz rechts.
- Die Ordnung muss nicht nach dem ganzen Inhalt gehen: In der Item-Datenbank ist der Inhaltstyp **Item**, geordnet wird nach dem **Schlüssel** getNummer(). Zum Vergleichen castest du: `Item hier = (Item) b.getItem();`



10 Brot	**20** Seil	**30** Fackel	**35** Karte	**40** Schlüssel	**45** Laterne
50 Heiltrank	**60** Schild	**65** Bogen	**70** Schwert	**75** Kompass	**80** Drachenei

Abbildung 1: Die Item-Datenbank (Item-Nummern) und die Namen der Items

Aufgabe 1: Echt oder gefälscht?

PAPIER ★

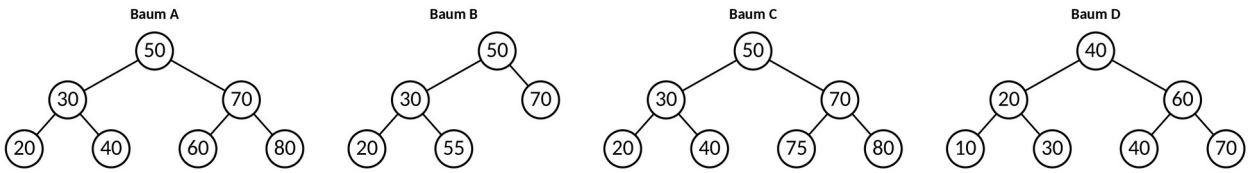


Abbildung 2: Vier Bäume aus dem Waldarchiv

a) Entscheide für jeden der Bäume A bis D, ob er ein binärer Suchbaum ist. Begründe jedes Nein mit einem Paar von Werten, das die Ordnung verletzt. **[4 BE]**

Baum	Suchbaum?	Begründung
A		
B		
C		
D		

b) Gib die Inorder der Bäume B und D an. Erläutere, wie du mit der Inorder prüfen kannst, ob ein Baum ein Suchbaum ist. **[2 BE]**

c) Gib alle Item-Nummern im Teilbaum mit der Wurzel 40 aus Abbildung 1 an. Begründe, zwischen welchen beiden Grenzen **jede** Nummer in diesem Teilbaum liegen muss – und welche Knoten diese Grenzen festlegen. **[2 BE]**

Infoteil 2: Suchen und Einfügen – immer nur ein Weg

Suchen: Vergleiche x mit der Wurzel. Gleich → gefunden. Kleiner → suche **nur** im linken Teilbaum weiter, größer → **nur** im rechten. Ist der Teilbaum leer, kommt x nicht vor. Der andere Teilbaum wird nie betreten – deshalb ist die Suche schnell.

```
public static boolean suche(BinTree b, int x) {
    if (b.isEmpty()) {
        return false;                       // hier müsste x stehen
    }
    int wert = (Integer) b.getItem();
    if (x == wert) {
        return true;
    }
    if (x < wert) {
        return suche(b.getLeft(), x);       // nur links weitersuchen
    }
    return suche(b.getRight(), x);          // nur rechts weitersuchen
}
```

Dasselbe geht ohne Rekursion: Eine Variable k „wandert“ durch den Baum.

```
public static boolean sucheIterativ(BinTree b, int x) {
    BinTree k = b;                          // k wandert durch den Baum
    while (!k.isEmpty()) {
        int wert = (Integer) k.getItem();
        if (x == wert) {
            return true;
        }
        if (x < wert) {
            k = k.getLeft();
        } else {
            k = k.getRight();
        }
    }
    return false;
}
```

Einfügen: Suche nach x. Wo die Suche auf einen **leeren Teilbaum** trifft, gehört x hin: `setItem(x)` macht den leeren Teilbaum zum Blatt. Ist x schon da, passiert nichts.

```
public static void einfuegen(BinTree b, int x) {
    if (b.isEmpty()) {
        b.setItem(x);                       // leerer Teilbaum wird Blatt
        return;
    }
    int wert = (Integer) b.getItem();
    if (x < wert) {
        einfuegen(b.getLeft(), x);
    } else if (x > wert) {
        einfuegen(b.getRight(), x);
    }                                       // x == wert: Duplikat, nichts tun
}
```

Aufwand: Wir zählen die **untersuchten Knoten** (nicht leere Knoten, deren Wert mit x verglichen wird). Der **Suchweg** ist die Folge dieser Knoten; endet er im Leeren, schreiben wir „→ leer“.

Aufgabe 2: Suchen mit Zählwerk

PAPIER ★

a) Gib für die Item-Datenbank (Abbildung 1) den Suchweg, die Anzahl der untersuchten Knoten und die Rückgabe von `suche(b, x)` an. **[3 BE]**

x	Suchweg	untersuchte Knoten	suche(b, x)
65			
33			
80			
50			
99			
5			

b) Nach jedem der 12 Items wird genau einmal gesucht. Bestimme, wie viele Knoten dabei im Durchschnitt untersucht werden. Gib außerdem an, wie viele Knoten bei **irgendeiner** Nummer höchstens untersucht werden, und begründe den Zusammenhang mit der Höhe des Baums. **[3 BE]**

c) Vergleiche mit der linearen Suche in einer Reihung, in der die 12 Items in zufälliger Reihenfolge stehen. **[1 BE]**

Aufgabe 3: Neue Items für die Datenbank

PAPIER ★★

a) In die Item-Datenbank (Abbildung 1) werden nacheinander die Items 55 (Kristall) und 85 (Zauberstab) eingefügt. Gib jeweils den Suchweg bis zur freien Stelle an und zeichne den Baum danach. **[2 BE]**

b) Danach wird `einfuegen(b, 30)` aufgerufen. Beschreibe, was passiert. **[1 BE]**

c) Dieselben 12 Items werden in anderer Reihenfolge in einen leeren Baum eingefügt (siehe unten). Entscheide jeweils, ob genau der Baum aus Abbildung 1 entsteht. Zeichne den Baum zu der Reihenfolge, bei der ein anderer Baum entsteht, und gib seine Höhe an. **[3 BE]**

(1) `50, 70, 30, 80, 60, 20, 40, 75, 65, 45, 35, 10`

(2) `50, 30, 35, 40, 20, 70, 60, 65, 80, 75, 10, 45`

d) Formuliere eine Regel, wann eine Einfügereihenfolge genau den Baum aus Abbildung 1 erzeugt. **[2 BE]**

Aufgabe 4: Struktogramm ↔ Java: die Item-Suche

PAPIER → IDE ★★

Die Item-Datenbank speichert **Item**-Objekte (Klassenkarte im Baumzeichner: `+ getNummer(): Ganzzahl`, `+ getName(): Zeichenkette`, `+ beschreibung(): Zeichenkette`). Geordnet wird nach der Nummer.

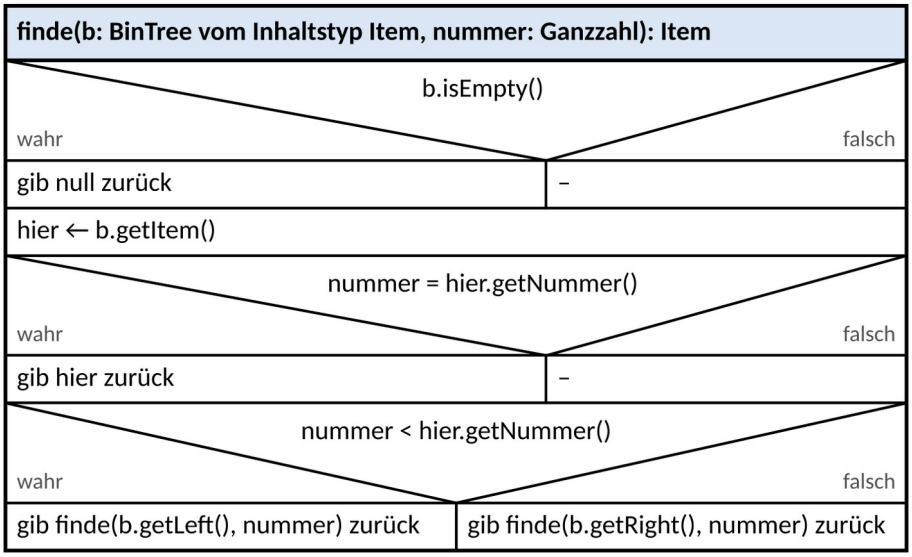
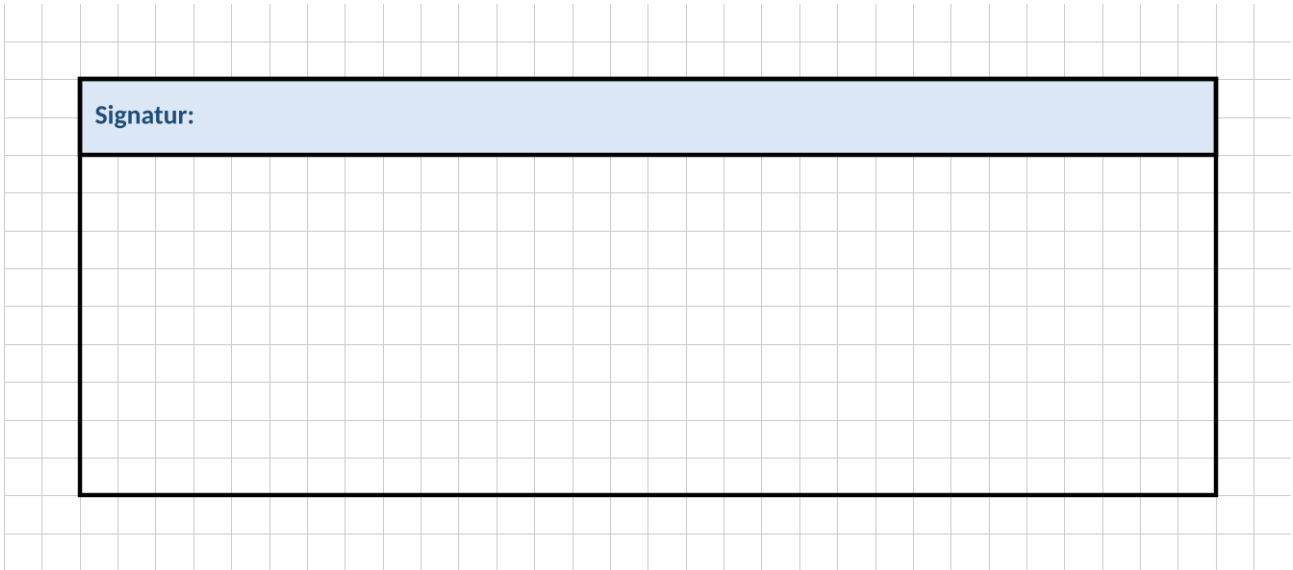


Abbildung 3: Die Operation finde als Struktogramm

a) Implementiere die Operation finde aus Abbildung 3 als static-Methode in Java. Denke an den Cast. **[4 BE]**

b) Gib an, was `finde(b, 65).beschreibung()` und `finde(b, 33)` für die Item-Datenbank liefern. **[1 BE]**

c) Entwickle zur Methode sucheIterativ (Infoteil 2) ein Struktogramm nach den Abi-Konventionen. **[3 BE]**



d) **Papier-Compiler:** Schreibe `suche` und `einfuegen` aus Level 5.4 (Basis) **ohne** Blick auf Infoteil 2 vollständig von Hand. Tippe sie dann unverändert ab und starte `L54SuchbaumTest`.

Regeln des Papier-Compilers

1. Schreibe die Methode **vollständig von Hand** – ohne Gerät, ohne Spickzettel außer den Werkzeugkarten.
2. Gehe den Abi-Check durch (Kasten am Ende von Teil 2).
3. Tippe deine Lösung **unverändert** ab – Zeichen für Zeichen, auch mit Fehlern!
4. Starte den Testrunner: Die Tests `basis_suche`, `basis_suche_nurEinWeg`, `basis_einfuegen` und `basis_einfuegen_leer` müssen grün sein.
5. Jeder Fehler kommt ins Fehlerlogbuch – erst dann korrigierst du in der IDE.

Infoteil 3: Löschen in drei Fällen

Beim Löschen muss der Baum ein Suchbaum **bleiben**. Zuerst wird der Knoten mit x gesucht. Dann gibt es drei Fälle:

1. **Blatt:** Der Knoten wird einfach entfernt – an seiner Stelle steht ein leerer Teilbaum (`setEmpty()`).
2. **Genau ein Kind:** Das Kind rückt **mit seinem ganzen Teilbaum** an die Stelle des gelöschten Knotens.
3. **Zwei Kinder:** Der Wert wird durch seinen **Nachfolger** ersetzt – den **kleinsten Wert im rechten Teilbaum** (einmal rechts, dann immer links). Der Nachfolger hat kein linkes Kind; er wird an seiner alten Stelle nach Fall 1 oder 2 entfernt. **Alternativ** nimmt man den **Vorgänger**, den größten Wert im linken Teilbaum (einmal links, dann immer rechts).

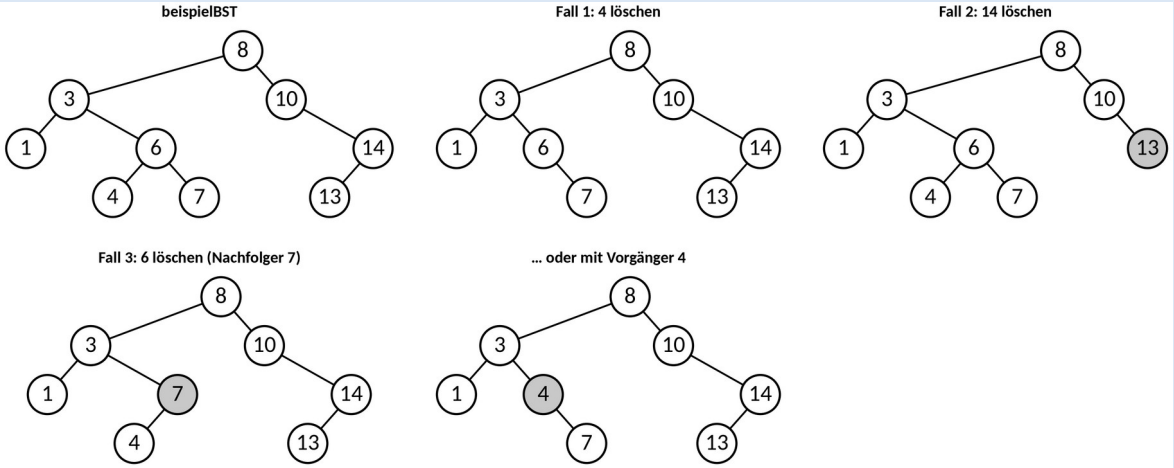


Abbildung 4: Die drei Fälle am beispielBST (grau: der Knoten, der nachgerückt ist)

Probe: Die Inorder nach dem Löschen ist die alte Inorder **ohne** x. Gibt eine Aufgabe nicht vor, ob Nachfolger oder Vorgänger, nimm den Nachfolger und schreib dazu, welche Variante du gewählt hast.

Aufgabe 5: Ein Item verschwindet

PAPIER ★★

Jedes Mal wird aus der **ursprünglichen** Item-Datenbank (Abbildung 1) gelöscht.

a) Lösche 10 (Brot) und – unabhängig davon – 60 (Schild). Gib jeweils den Fall an und zeichne nur den veränderten Teil des Baums. **[2 BE]**

b) Lösche 30 (Fackel) einmal mit dem Nachfolger und einmal mit dem Vorgänger. Zeichne beide Ergebnisse. **[3 BE]**

c) Lösche die Wurzel 50 (Heiltrank) mit dem Nachfolger. Gib den Nachfolger an, zeichne den Baum und führe die Inorder-Probe durch. **[3 BE]**

d) Begründe, dass der Nachfolger eines Knotens mit zwei Kindern kein linkes Kind hat und Fall 3 deshalb immer auf Fall 1 oder 2 zurückgeführt werden kann. **[2 BE]**

Aufgabe 6: Fehlerjagd in der Datenbank

PAPIER ★★

Jede der drei Methoden enthält einen typischen Fehler. „Item-Datenbank“ meint den Baum aus Abbildung 1 mit Ganzzahlen.

(A) einfuegen soll x in den Suchbaum einfügen.

```
1  public static void einfuegen(BinTree b, int x) {
2      if (b.isEmpty()) {
3          b = new BinTree(x);
4          return;
5      }
6      int wert = (Integer) b.getItem();
7      if (x < wert) {
8          einfuegen(b.getLeft(), x);
9      } else if (x > wert) {
10         einfuegen(b.getRight(), x);
11     }
12 }
```

a) Gib an, wie die Item-Datenbank nach `einfuegen(b, 55)` aussieht und was beim Einfügen in einen leeren Baum geschieht. Erläutere die Ursache und korrigiere die Methode. **[3 BE]**

(B) minimum soll den kleinsten Wert liefern (leerer Baum: −1).

```
1  public static int minimum(BinTree b) {
2      if (b.isEmpty()) {
3          return -1;
4      }
5      if (b.isLeaf()) {
6          return (Integer) b.getItem();
7      }
8      return minimum(b.getLeft());
9  }
```

b) Gib die Rückgabe für die Item-Datenbank, den beispielBST, den Baum `entartet()` (1 bis 5 eingefügt) und den Baum 5(2(-,3),9) an. Korrigiere die Methode und erläutere, warum Tests nur mit der Item-Datenbank den Fehler nicht finden. **[3 BE]**

(C) sucheIterativ soll wie im Infoteil 2 arbeiten.

```
1  public static boolean sucheIterativ(BinTree b, int x) {
2      BinTree k = b;
3      while (!k.isLeaf()) {
4          int wert = (Integer) k.getItem();
5          if (x == wert) {
6              return true;
7          }
8          if (x < wert) {
9              k = k.getLeft();
10         } else {
11             k = k.getRight();
12         }
13     }
14     return false;
15 }
```

c) Gib für die Item-Datenbank die Rückgabe bzw. den Laufzeitfehler für x = 50, 65 und 99 an und ebenso für den leeren Baum. Korrigiere die Methode. **[3 BE]**

Aufgabe 7: IDE-Level 5.4 – Suchbaum (Basis und Erweiterung)

IDE ★ – ★★

Klasse **Suchbaum** (alle Methoden static), Testklasse `L54SuchbaumTest`. Die Bäume enthalten Ganzzahlen ≥ 0; Duplikate werden ignoriert.

Stufe	Methoden (Signatur wie im Abitur)	Tests beginnen mit
★ Basis	suche(b: BinTree vom Inhaltstyp Ganzzahl, x: Ganzzahl): Wahrheitswert – rekursiv minimum(b): Ganzzahl (leer: −1) einfuegen(b, x: Ganzzahl): nichts – rekursiv	`basis_`
★★ Erweiterung	sucheIterativ(b, x): Wahrheitswert – mit Schleife vergleiche(b, x): Ganzzahl – untersuchte Knoten baueAus(r: Reihung von Ganzzahl): BinTree	`erweiterung_`

- `basis_suche_nurEinWeg` sucht absichtlich in einem **ungültigen** Baum 8(3,10(5,-)) nach 5: Eine echte Suchbaum-Suche geht nach links und findet die 5 **nicht**. Wer beide Teilbäume durchsucht (wie enthaelt aus Level 5.2), wird rot.
- baueAus erzeugt **einen** leeren Baum (`new BinTree()`, nicht null) und fügt die Werte der Reihe nach ein. Mit Aufgabe 3 c) kannst du deine Ergebnisse prüfen.
- **Baumzeichner** (`AQ_W5_Baumzeichner_Geruest.txt`): Aufgabe 4 `sucheZeigen` (Suchweg einfärben), Aufgabe 5 `einfuegenZeigen`, Aufgaben 6 bis 8 Klasse **ItemDatenbank** (einfuegen, finde, inventar) mit den Items aus Abbildung 1 – eure Lösung aus Aufgabe 4 a) passt direkt hinein.

> 🎯 **Im Lab üben: Level 5.4** „Die Item-Datenbank – binäre Suchbäume“ – Punkte gibt es dort automatisch.

Teil 2 – Ausgeglichen oder entartet? (DS 35)

Infoteil 4: Die Höhe entscheidet

Die Suche untersucht höchstens so viele Knoten, wie der Baum **hoch** ist. Dieselben Werte können aber – je nach Einfügereihenfolge – sehr verschieden hohe Bäume ergeben:

- **Entartet:** Jeder innere Knoten hat nur ein Kind – der Baum ist eine „Wäscheleine“ und nicht besser als eine Liste. Höhe = Anzahl der Knoten n. Das passiert z. B., wenn sortierte Werte eingefügt werden (`entartet()`: 1, 2, 3, 4, 5).
- **Ausgeglichen:** Der Baum ist möglichst niedrig. Ein **vollständiger** Baum mit k Ebenen hat 1 + 2 + 4 + … + 2^(k−1) = **2^k − 1** Knoten.

Merke: Verdoppelt sich die Anzahl der Knoten (aus 2^k − 1 werden 2^(k+1) − 1), kommt im ausgeglichenen Baum **genau eine Ebene** dazu – die Suche braucht **einen** Vergleich mehr. Im entarteten Baum verdoppelt sich die Anzahl der Vergleiche.

Aufgabe 8: Die Wäscheleine

PAPIER ★★

a) Die 12 Items aus Abbildung 1 werden aufsteigend nach Nummer eingefügt (10, 20, 30, …, 80). Beschreibe den entstehenden Baum, gib seine Höhe an und bestimme, wie viele Knoten im Durchschnitt untersucht werden, wenn nach jedem Item genau einmal gesucht wird. Vergleiche mit Aufgabe 2 b). **[2 BE]**

b) Ergänze die Tabelle. Die kleinstmögliche Höhe ist zugleich die größte Anzahl untersuchter Knoten im ausgeglichenen Baum. **[3 BE]**

Anzahl n der Werte	7	15	63	127	255	1.000	1.023	1.000.000
Höhe bei sortiertem Einfügen								
kleinstmögliche Höhe								

c) Begründe die Regel „Verdopplung der Anzahl → ein Vergleich mehr“ am vollständigen Baum. Beurteile, was das für eine Item-Datenbank mit 1 000 000 Items bedeutet. **[2 BE]**

Aufgabe 9: Mitte nach oben

PAPIER ★★★

Aus einer **aufsteigend sortierten** Reihung soll ein möglichst niedriger Suchbaum entstehen: Das mittlere Element wird die Wurzel, die linke Hälfte der linke Teilbaum, die rechte Hälfte der rechte Teilbaum – rekursiv (Teile und herrsche aus Welt 3).

```
1  public static BinTree ausgeglichenAus(int[] sortiert) {
2      return ausgeglichenAus(sortiert, 0, sortiert.length - 1);
3  }
4
5  public static BinTree ausgeglichenAus(int[] r, int lo, int hi) {
6      if (lo > hi) {
7          return new BinTree();                    // Bereich leer -> leerer Baum
8      }
9      int mitte = (lo + hi) / 2;
10     BinTree b = new BinTree(r[mitte]);
11     b.setLeft(ausgeglichenAus(r, lo, mitte - 1));
12     b.setRight(ausgeglichenAus(r, mitte + 1, hi));
13     return b;
14 }
```

a) Dokumentiere den Aufruf `ausgeglichenAus(r)` mit r = {10, 20, 30, 40, 50, 60, 70} in einer Rekursionstabelle. Trage die Aufrufe von ausgeglichenAus(r, lo, hi) in der Reihenfolge ein, in der sie **beginnen**. **[4 BE]**

Rekursionstiefe *)	lo	hi	mitte	liefert
0	0	6		

*) Bei jedem rekursiven Aufruf erhöht sich die Rekursionstiefe um 1. Der Erstaufruf hat die Rekursionstiefe 0.

b) Zeichne den entstandenen Baum. Gib die Anzahl der Aufrufe an und begründe, dass für eine Reihung mit n Werten immer 2n + 1 Aufrufe entstehen. **[2 BE]**

c) Wende ausgeglichenAus auf die 12 sortierten Item-Nummern (10, 20, 30, 35, 40, 45, 50, 60, 65, 70, 75, 80) an. Zeichne den Baum, gib seine Höhe an und bestimme die Anzahl der untersuchten Knoten bei der Suche nach 65. **[3 BE]**

Aufgabe 10: Die isBST-Falle

PAPIER ★★★

Das Team Eisenhut will prüfen, ob ein Baum ein Suchbaum ist. Es vergleicht jeden Knoten mit seinen **Kindern**:

```
1  public static boolean nurKinder(BinTree b) {
2      if (b.isEmpty()) {
3          return true;
4      }
5      int w = (Integer) b.getItem();
6      if (!b.getLeft().isEmpty() && (Integer) b.getLeft().getItem() >= w) {
7          return false;
8      }
9      if (!b.getRight().isEmpty() && (Integer) b.getRight().getItem() <= w) {
10         return false;
11     }
12     return nurKinder(b.getLeft()) && nurKinder(b.getRight());
13 }
```

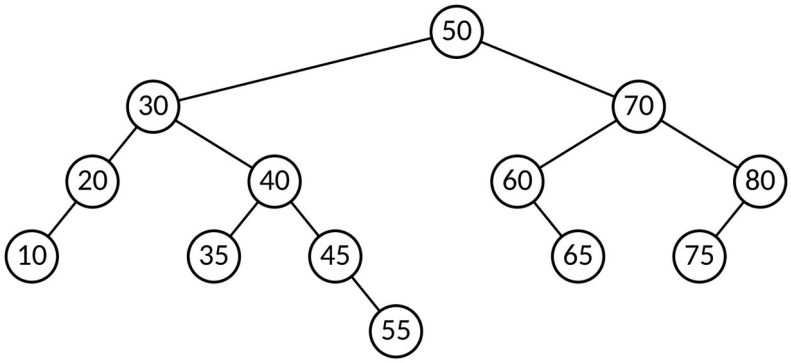


Abbildung 5: Der Baum F (die Item-Datenbank, nachdem jemand die 55 von Hand angehängt hat)

a) Gib `nurKinder(F)` an. Erläutere, warum F trotzdem kein Suchbaum ist, und zeige die Folge: Gib den Suchweg von `suche(F, 55)` und die Rückgabe an. **[3 BE]**

b) Gib einen Baum mit **drei** Knoten an, für den nurKinder wahr liefert, obwohl er kein Suchbaum ist. **[1 BE]**

Die richtige Prüfung gibt jedem Teilbaum die **Grenzen** mit, die alle Vorfahren vorgeben (exklusiv: min < Wert < max). Da alle Werte ≥ 0 sind, steht −1 für „keine Grenze“.

```
1  public static boolean istBST(BinTree b) {
2      return istBST(b, -1, -1);               // -1: keine Grenze
3  }
4
5  public static boolean istBST(BinTree b, int min, int max) {
6      if (b.isEmpty()) {
7          return true;
8      }
9      int wert = (Integer) b.getItem();
10     if (wert <= min) {                      // muss echt größer als min sein
11         return false;
12     }
13     if (max != -1 && wert >= max) {         // muss echt kleiner als max sein
14         return false;
15     }
16     return istBST(b.getLeft(), min, wert) && istBST(b.getRight(), wert, max);
17 }
```

c) Dokumentiere den Aufruf `istBST(F)` in einer Rekursionstabelle (Aufrufe von istBST(b, min, max) in der Reihenfolge ihres Beginns; „–“ für −1, „leer“ für einen leeren Teilbaum). Beachte: `&&` wertet den rechten Aufruf nur aus, wenn der linke wahr geliefert hat. **[4 BE]**

Rekursionstiefe *)	Knoten	min	max	Rückgabe
0	50	–	–	

**) Bei jedem rekursiven Aufruf erhöht sich die Rekursionstiefe um 1. Der Erstaufruf hat die Rekursionstiefe 0.*

d) Erläutere, warum für min der Wert −1 ohne Sonderbehandlung als „keine Grenze“ funktioniert, für max aber nicht. **[1 BE]**

Aufgabe 11: IDE-Level 5.4 – Suchbaum (Experte) IDE ★★★

Weiter in der Klasse **Suchbaum**, Testklasse `L54SuchbaumTest` (Tests beginnen mit `experte_`).

Methode	Hinweis
istBST(b): Wahrheitswert istBST(b, min: Ganzzahl, max: Ganzzahl): Wahrheitswert	wie in Aufgabe 10 – die öffentliche Methode ruft `istBST(b, -1, -1)` auf
ausgeglichenAus(sortiert: Reihung von Ganzzahl): BinTree ausgeglichenAus(r, lo: Ganzzahl, hi: Ganzzahl): BinTree	wie in Aufgabe 9; leere Reihung → leerer Baum (nicht null)
nachfolger(b, x: Ganzzahl): Ganzzahl	kleinster Wert **größer als** x, −1 wenn keiner; x muss nicht im Baum stehen. beispielBST: nachfolger(8) = 10, nachfolger(7) = 8

Abi-Check Suchbäume

- ☐ Suchen und Einfügen: in jedem Schritt **nur ein** Teilbaum – kein zweiter rekursiver Aufruf.
- ☐ Basisfall `if (b.isEmpty())` zuerst; beim Einfügen **setItem** auf dem leeren Teilbaum (nicht `b = new BinTree(x)`).
- ☐ Inhalt casten: `(Integer) b.getItem()` bzw. `(Item) b.getItem()`, verglichen wird der **Schlüssel**.
- ☐ Duplikate: Was soll passieren? (bei uns: ignorieren)
- ☐ Löschen: Fall angeben; bei zwei Kindern Nachfolger **oder** Vorgänger nennen; Inorder-Probe.
- ☐ Suchbaum prüfen: **Grenzen** aller Vorfahren, nicht nur die Kinder – oder Inorder aufsteigend.
- ☐ Aufwand: hängt von der **Höhe** ab – ausgeglichen etwa $\log_2(n)$, entartet n.

🎯 **Im Lab üben: Level 5.5** „Krumm oder buschig? – ausgeglichen, entartet, isBST“ – Punkte gibt es dort automatisch.

Für Schnelle

- **Löschen implementieren:** Schreibe `loesche(b: BinTree vom Inhaltstyp Ganzzahl, x: Ganzzahl): nichts` mit den drei Fällen aus Infoteil 3 (Fall 2: Inhalt und beide Teilbäume des Kindes übernehmen; Fall 3: Nachfolger mit einer Hilfsmethode bestimmen). Teste im Hauptprogramm mit `Testhilfe.klammer` und den Bäumen aus Aufgabe 5.
- **Struktogramm:** Stelle ausgeglichenAus(r, lo, hi) aus Aufgabe 9 als Struktogramm nach EH § 5 dar.
- Bestimme von Hand nachfolger(45), nachfolger(80) und nachfolger(33) in der Item-Datenbank.